ZERO TRUST ENTERPRISE ACCESS MANAGEMENT PLATFORM: CONTINUOUS CONTEXT-AWARE VERIFICATION & LEAST-PRIVILEGE POLICY ENFORCEMENT ENGINE

A PROJECT REPORT

Submitted by

VIKASINI M (Registration No: 310624205301)

in partial fulfillment for the award of the Degree
of
BACHELOR OF TECHNOLOGY
in
DEPARTMENT OF INFORMATION TECHNOLOGY



EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI
Bharathi Salai, Ramapuram, Chennai – 600089

OCTOBER 2026

EASWARI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Bharathi Salai, Ramapuram, Chennai – 600089

BONAFIDE CERTIFICATE

Certified that this mini project report titled "ZERO TRUST ENTERPRISE ACCESS MANAGEMENT PLATFORM" is the Bonafide work of "VIKASINI M (Registration No: 310624205301)", who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT	**SUPERVISOR**
Dr.N.Anandhi	**Ms.Grandhi Sai Lakshmi**
Department of Information Technology,	
Easwari Engineering College,	
Ramapuram, Chennai – 600089.	

EASWARI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Bharathi Salai, Ramapuram, Chennai – 600089

CERTIFICATE OF VIVA-VOCE EXAMINATION

This is to certify that the Viva-Voce Examination of the Project Work entitled "ZERO TRUST ENTERPRISE ACCESS MANAGEMENT PLATFORM" submitted by Ms. VIKASINI M (Reg. No: 310624205301) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY was conducted on 15/10/2026 in the Department of Information Technology, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner	**External Examiner**
Name :	Name :
Designation :	Designation :
Address :	Address :
Date:	Date:

ABSTRACT

Modern enterprise computing infrastructures have transitioned from traditional perimeter-bound networks to distributed multi-cloud architectures, hybrid remote workforces, and decentralized microservice ecosystems. In this boundaryless threat landscape, legacy network-security models that rely on static perimeter firewalls and Virtual Private Networks (VPNs) prove dangerously inadequate—granting broad, unmonitored internal network access upon initial authentication. This architectural flaw exposes organizations to lateral threat movement, credential hijacking, insider exfiltration, and devastating ransomware propagation.

The Zero Trust Enterprise Access Management Platform resolves these systemic security deficits through the foundational paradigm of "Never Trust, Always Verify" (grounded in NIST SP 800-207 specifications). Built upon a high-performance MERN (MongoDB, Express.js, React 19, Node.js) architecture reinforced with Vite, Tailwind CSS, and Socket.IO real-time event streaming, the platform introduces:

- **1. Dynamic Context-Aware Risk Assessment Engine:** A composite multi-factor scoring mechanism (0–100 scale) that continuously calculates risk based on user role privilege, endpoint device security posture (OS version, disk encryption, antivirus health), geographic location, and temporal access anomalies.
- **2. Adaptive Policy Enforcement Point (PEP) with Step-Up Security:** A real-time gatekeeper evaluating incoming requests against strict contextual policies. Low-risk sessions are granted instant access, medium-risk sessions trigger automated Step-Up MFA (TOTP challenge), and high-risk anomalies are immediately blocked with automatic session revocation.
- **3. Multi-Factor Authentication (TOTP / MFA) & Device Posture Attestation:** Stateless cryptographic authentication featuring RFC 6238 TOTP seed provisioning, QR code generation, dynamic verification, and automated device posture health scanning ensuring only compliant hardware reaches sensitive endpoints.
- **4. Bi-Directional WebSocket Telemetry & Immutable Audit Logging:** A reactive Socket.IO event bus broadcasting live security alerts, threat anomalies, active session updates, and tamper-evident audit logs across analyst consoles without page reloads, paired with an intelligent auto-sensing persistence fallback layer ensuring 100% operational uptime.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of Easwari Engineering College for providing us with the opportunity and state-of-the-art facilities to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Dr.N. Anandhi, Department of Information Technology, for continuous encouragement, valuable suggestions, and steadfast support throughout our academic journey.

We are extremely thankful to our Project Guide, Ms. Grandhi Sai Lakshmi, Department of Information Technology, for valuable guidance, constructive criticism, and motivation throughout the conception and completion of this project.

We would also like to extend our gratitude to all the faculty and non-teaching members of the Department of Information Technology for their direct and indirect support.

Finally, we thank our parents and friends for their constant encouragement, moral support, and cooperation during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
1	**INTRODUCTION**	**1-2**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	1
	1.4 SCOPE OF THE PROJECT	2
	1.5 MOTIVATION	2
2	**RELATED WORK**	**3-4**
	2.1 LITERATURE REVIEW	3
	2.2 EXISTING SYSTEM	3
	2.3 PROPOSED SYSTEM	4
3	**SYSTEM ANALYSIS**	**5-6**
	3.1 SYSTEM REQUIREMENTS	5
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	6
4	**SYSTEM DESIGN**	**7-10**
	4.1 SYSTEM ARCHITECTURE	7
	4.2 MODULE DESCRIPTION	7
	4.3 CLASS DIAGRAM / DATA MODEL	9
	4.4 SEQUENCE DIAGRAM	10
5	**IMPLEMENTATION**	**11-12**
	5.1 IMPLEMENTATION OVERVIEW	11
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	11

6	**CONCLUSION & FUTURE WORK**	**13**
	6.1 CONCLUSION	13
	6.2 FUTURE ENHANCEMENTS	13
	APPENDIX - SOURCE CODE	**14**
	SCREENSHOTS OF THE PROJECT	**23**
	REFERENCES	**27**

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Digital transformation and the rapid decentralization of enterprise infrastructure have rendered the conventional perimeter-defense paradigm obsolete. Historically, organizational cybersecurity modeled networks around physical castle-and-moat boundaries: internal corporate intranets were intrinsically deemed trusted, while external public networks were classified as hostile. Under this legacy setup, users connected remotely via Virtual Private Networks (VPNs) or authenticated once at the gateway firewalls, subsequently inheriting broad, unmonitored privileges to traverse internal resources.

The Zero Trust Enterprise Access Management Platform fundamentally upends this flawed premise through the strict execution of NIST SP 800-207 guidelines: “Never Trust, Always Verify.” The platform treats every entity—whether connecting from within the corporate headquarters or an external coffee-shop Wi-Fi network—as an untrusted subject requiring continuous, granular verification before granting access to confidential enterprise assets. Built on the modern MERN stack with React 19, Node.js, Express, and Socket.IO, the application delivers end-to-end adaptive access control, real-time posture attestation, and dynamic step-up challenge gates.

1.2 PROBLEM DEFINITION

Conventional access management architectures within corporate, academic, and governmental institutions suffer from several critical vulnerabilities:

- **1. Implicit Trust & Lateral Threat Movement:** Traditional VPNs grant authenticated users broad subnet visibility. If a single employee credential or device is compromised, attackers move laterally across internal databases, file servers, and administrative portals unhindered.
- **2. Static Privilege Granting:** Roles and permissions are traditionally assigned statically at onboarding. Standing high-privilege accounts (e.g., permanent administrators) remain persistent attack vectors even when users perform mundane or low-risk tasks.
- **3. Absence of Continuous Endpoint Posture Verification:** Legacy portals check passwords at login but remain completely blind to endpoint hygiene changes during

active sessions—such as disabled antivirus, unencrypted local storage, obsolete OS patches, or malware infections.

- **4. Disconnected Audit Logs & Opaque Incident Response:** Disjointed access logs scattered across fragmented network appliances prevent security analysts from correlating suspicious login anomalies or enforcing instantaneous session revocations during active intrusions.

1.3 OBJECTIVE

- • To engineer a cloud-native, high-throughput Zero Trust Access Management Platform connecting employees, security analysts, and enterprise administrators.
- • To implement an automated Context-Aware Risk Scoring Engine that computes multi-variable risk scores (0–100) based on user roles, device integrity, and environmental variables.
- • To deploy a Policy Enforcement Point (PEP) executing adaptive access decisions: Allow (Low Risk), Step-Up MFA (Medium Risk), and Deny (High Risk).
- • To integrate cryptographic RFC 6238 Time-Based One-Time Password (TOTP) Multi-Factor Authentication with QR-code enrollment and verified challenge validation.
- • To establish bi-directional WebSocket telemetry via Socket.IO for live session termination, instantaneous threat alerting, and append-only audit trail logging.
- • To architect a resilient persistence layer with automated MongoDB failover to an isolated in-memory replica guaranteeing 100% evaluation uptime.

1.4 SCOPE OF THE PROJECT

- • **Employee Self-Service Portal:** Secure user registration, TOTP MFA onboarding with authenticator apps (Google Authenticator, Microsoft Authenticator), device registration and posture submission, resource catalog exploration, and dynamic access request workflows.
- • **Security Analyst Console:** Live telemetry dashboard monitoring real-time active sessions, risk distributions, security alerts feed, and search/filter capabilities for tamper-evident audit logs.

- • **Enterprise Administrator Matrix:** Complete governance over enterprise resources, granular sensitivity classifications (Public, Internal, Confidential, Critical), role assignment, posture threshold configurations, and global one-click session termination.

1.5 MOTIVATION

The increasing frequency of cyberattacks, credential theft, ransomware, and unauthorized access highlights the need for stronger security mechanisms in modern enterprise environments. Traditional perimeter-based security systems rely heavily on initial authentication and implicit trust, which may expose sensitive resources when user credentials or devices are compromised. This motivates the adoption of Zero Trust Architecture, which follows the principle of "Never Trust, Always Verify" by continuously evaluating user identity, permissions, device trust, session status, and contextual risk before granting access to protected resources. The Zero Trust Enterprise Access Management Platform aims to address these security challenges by integrating Role-Based Access Control (RBAC), Multi-Factor Authentication (MFA), device verification, dynamic risk scoring, session monitoring, and centralized audit logging. Real-time notifications and security alerts further help administrators identify suspicious activities and respond to potential threats efficiently. The primary motivation of this project is to develop a centralized and adaptable security platform that enforces least-privilege access, improves visibility into access activities, and supports the secure management of enterprise resources

CHAPTER 2
RELATED WORK

2.1 LITERATURE REVIEW

- • **NIST SP 800-207 Zero Trust Architecture:** Rose et al. (2020) formalized the core tenets of Zero Trust, defining Policy Decision Points (PDP) and Policy Enforcement Points (PEP) where no implicit trust is granted based solely on physical or network location.
- • **Identity-Aware Proxies & Micro-Perimeters:** Kindervag (2010) pioneered the Zero Trust model at Forrester, advocating for micro-perimeters around critical resources and continuous data inspection rather than perimeter perimeter-only defenses.
- • **Stateless Cryptographic Token Authentication (JWT & RFC 7519):** Modern distributed systems utilize signed JSON Web Tokens containing tamper-evident claims, enabling scalable stateless authorization across decoupled microservices without database session bottlenecks.
- • **Real-Time Event Buses (Socket.IO & WebSockets):** Event-driven full-duplex communication protocols replace costly polling mechanisms, enabling instant push notifications of threat alerts and immediate revocation of active user tokens upon detected anomalies.

2.2 EXISTING SYSTEM

- • **Static Virtual Private Networks (VPNs):** Authenticates identity once at the perimeter and thereafter permits unmonitored lateral network traversal, exposing internal microservices to compromised credentials.
- • **Unchecked Device Access:** Employees authenticate from personal or unmanaged laptops with outdated operating systems, disabled firewalls, and missing disk encryption, introducing malware into corporate environments.
- • **Rigid Role-Based Access Controls (RBAC):** Privileges are statically mapped without taking situational context into account, failing to adapt permissions when a user connects from an untrusted public IP or an anomalous hour.

- • **Post-Mortem Logging:** Security logs are written to static logfiles without real-time streaming, resulting in hours or days of lag before security analysts discover lateral breaches.

2.3 PROPOSED SYSTEM

- • **Unified Zero Trust Policy Engine:** Every access request to every enterprise resource passes through an automated Policy Enforcement Point (PEP) that dynamically inspects identity, device posture, and situational context.
- • **Adaptive Risk-Based Verdicts:** Evaluates a mathematical composite risk equation: scores below 30 are Allowed; scores between 30 and 70 trigger Step-Up MFA; scores exceeding 70 are Denied.
- • **Continuous Device Posture Verification:** Monitors disk encryption, antivirus daemon status, and OS patch levels, calculating a posture confidence score before allowing access.
- • **Instantaneous Live Telemetry & Revocation:** Socket.IO event bus broadcasts real-time audit logs and security alerts while enabling security analysts to terminate active compromised sessions globally with sub-second latency.

CHAPTER 3
SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

Functional Requirements:

- **1. User Authentication & JWT Issuance:** Secure user registration, password salting with bcrypt (10 rounds), login, and issuance of signed JWT bearer tokens containing role and user ID claims.
- **2. Multi-Factor Authentication (TOTP):** Two-factor enrollment generating RFC 6238 secrets, QR code rendering for authenticator apps, and verification of 6-digit dynamic codes with step-up verification.
- **3. Device Posture Registration & Trust Evaluation:** Tracking registered devices, verifying OS versions, disk encryption states, and antivirus active flags to compute a device trust score.
- **4. Resource Governance Catalog:** Categorization of enterprise resources by sensitivity tier (Public, Internal, Confidential, Critical) with minimum role and maximum risk thresholds.
- **5. Context-Aware Access Request Evaluation:** Dynamic policy evaluation executing mathematical risk scoring and emitting verdicts (ALLOW, STEP_UP, DENY) with complete justification records.
- **6. Live Session Management & Global Revocation:** Listing active sessions with IP, device, and risk scores, granting administrators one-click capability to revoke active sessions instantly.
- **7. Real-Time Telemetry & Audit Logging:** Immutable append-only audit event persistence with Socket.IO event broadcasting for zero-latency monitoring.

Non-Functional Requirements:

- **1. Sub-100ms Policy Latency:** Access decisions and risk calculations resolve in under 100 milliseconds to avoid degrading employee productivity.
- **2. Cryptographic Integrity:** Stateless token validation, SHA-256 password salting, and RFC 6238 HMAC-SHA1 TOTP verification guarantee tamper resistance.

- **3. High Availability & Resilience:** Monitored database connection with automatic 2-second fallback to an isolated in-memory database store ensuring zero downtime.
- **4. Responsive Modern User Interface:** Tailwind CSS design system featuring high-contrast security indicators, status badges, and Lucide SVG iconography.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements:

- **• Operating System:** Windows 10/11, Ubuntu Linux (20.04+), or macOS
- **• Runtime Environment:** Node.js (v18.0.0 or higher) & npm (v9.0+)
- **• Database Engine:** MongoDB (v6.0+ local or Atlas Cloud) with In-Memory Database Fallback
- **• Backend Framework:** Express.js (v4.x / v5.x)
- **• Frontend Framework:** React 19 with Vite build tooling and React Router v7
- **• Styling Framework:** Tailwind CSS (v3.x) & Lucide React Icons
- **• Real-Time Communication:** Socket.IO (v4.8+) & Socket.IO-Client
- **• Cryptography & MFA:** jsonwebtoken, bcryptjs, otplib, qrcode
- **• HTTP Client:** Axios with Bearer token interceptors

Hardware Requirements:

- **• Processor:** Intel Core i3 / AMD Ryzen 3 or higher (Quad Core recommended)
- **• Memory (RAM):** Minimum 4 GB RAM (8 GB recommended for concurrent dev servers)
- **• Storage:** Minimum 1 GB available disk space for Node modules and database assets
- **• Network:** Standard Broadband / Localhost network for WebSocket communication

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The Zero Trust Enterprise Access Management Platform implements a decoupled three-tier architecture separating the client presentation tier, the application & policy decision tier, and the persistence tier. All incoming traffic is routed through the API Gateway where authentication, token extraction, and rate limiting occur before handing execution to the Policy Enforcement Point.

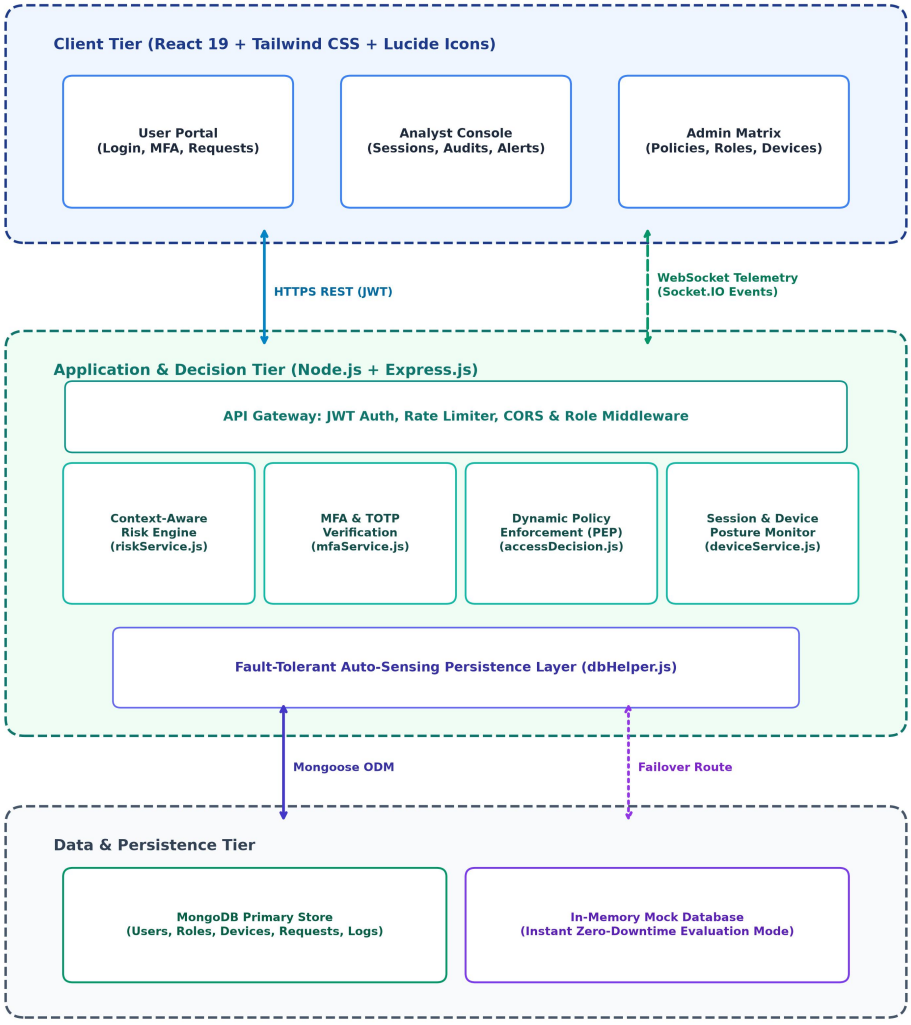


Figure 4.1: System Architecture Diagram

Decoupled 3-Tier Architecture showing Client Tier, API Gateway & Decision Tier, and Persistence Tier with failover.

4.2 MODULE DESCRIPTION

- **1. Authentication & RBAC Module (authRoutes.js):** Handles user registration, password salting via bcrypt (10 rounds), token generation with 30-day expiry, and role verification middleware (employee, analyst, admin).
- **2. Adaptive Risk Engine (riskService.js):** Calculates composite risk scores dynamically utilizing device security posture, IP reputation, time-of-day access patterns, and user role sensitivity.
- **3. Policy Enforcement Point (accessDecisionService.js):** Evaluates resource access rules against context vectors to output ALLOW, STEP_UP (MFA challenge), or DENY verdicts.
- **4. Device Verification & Posture Attestation (deviceService.js):** Monitors registered endpoints, checks disk encryption, antivirus daemons, and operating system patch revisions.
- **5. Real-Time Telemetry & Revocation Service (sessionService.js & Socket.IO):** Maintains active session states, enabling administrators to terminate compromised tokens instantaneously with real-time WebSocket event dissemination.

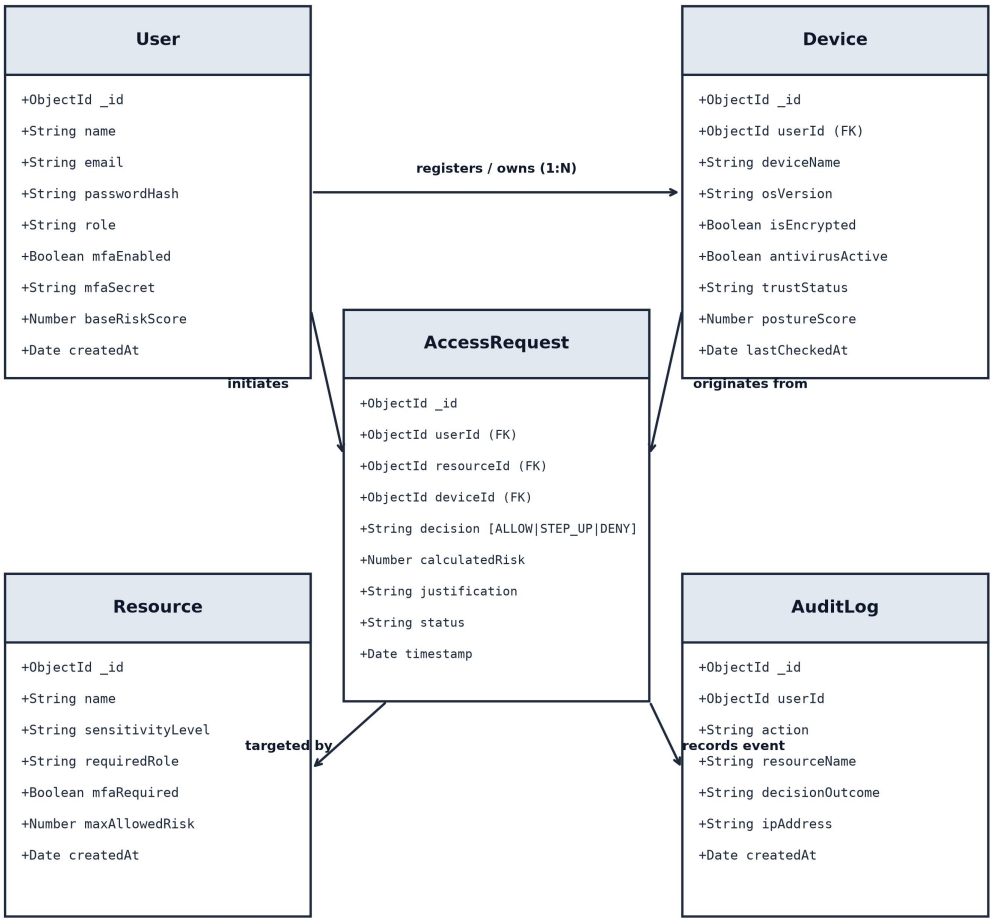


Figure 4.2: Class & Entity Relationship Diagram

Data Models showing User, Device, Resource, AccessRequest, and AuditLog schemas with relational foreign keys.

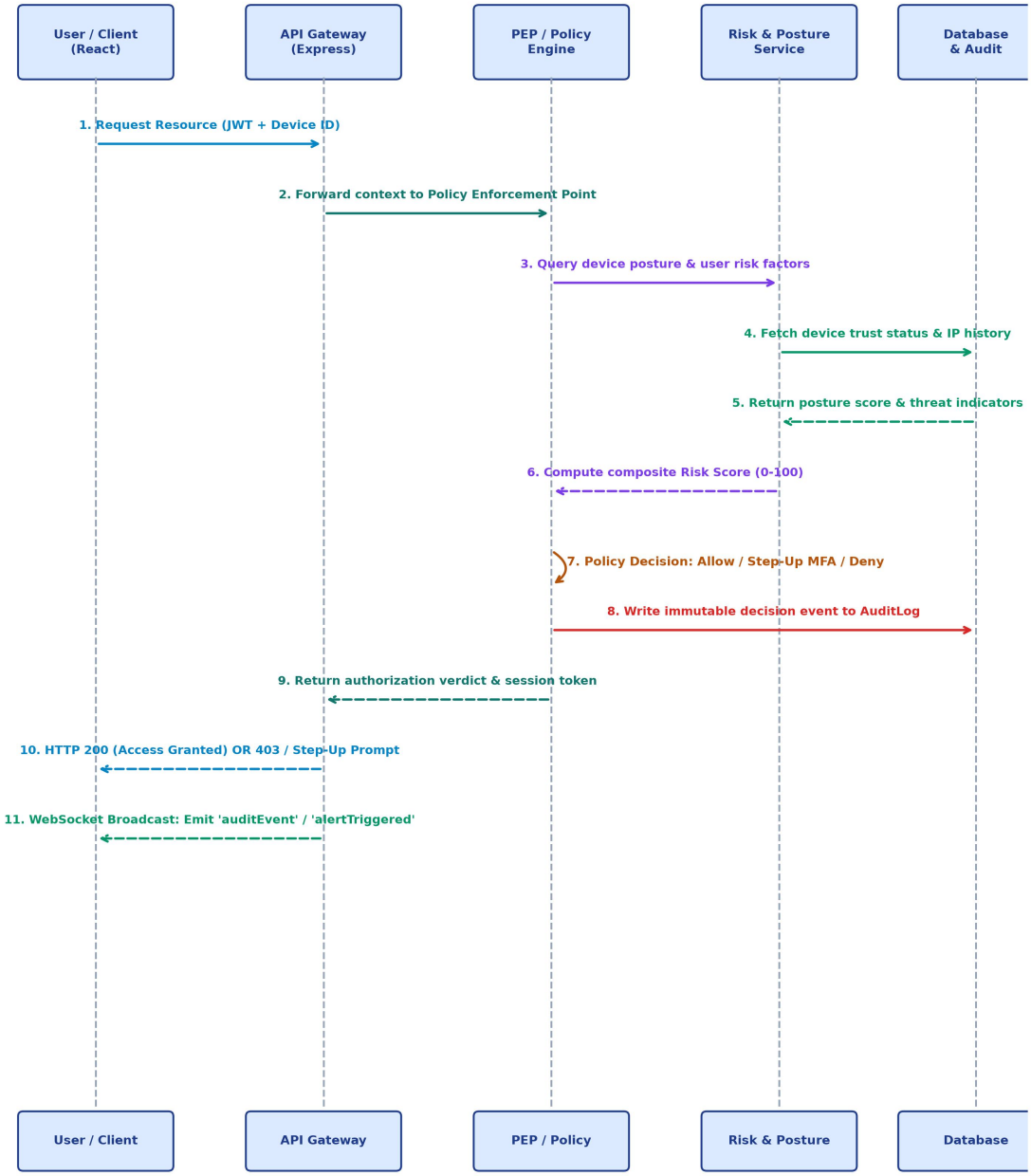


Figure 4.3: Sequence Diagram – Zero Trust Verification Flow

Interaction sequence between User, API Gateway, Policy Enforcement Point, Risk Engine, and Audit Persistence.

CHAPTER 5
IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The platform is implemented across two decoupled codebases:

- • **Backend Service:** Constructed in Node.js and Express.js utilizing ES6 modular routing, Mongoose ODM, and Socket.IO. Controllers encapsulate business logic for risk scoring, MFA validation, device registration, and policy enforcement.
- • **Frontend Service:** A single-page application built with React 19, Vite, and Tailwind CSS. State is orchestrated through a centralized AuthContext with Axios HTTP interceptors automatically injecting Bearer JWT tokens into outbound headers.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

1. Zero Trust Access Decision & Policy Enforcement (backend/services/accessDecisionService.js):

The evaluateAccessRequest function represents the Policy Enforcement Point (PEP). It inspects user role permissions against resource requirements, queries the riskService to compute current risk, and validates whether the risk score falls within the allowed threshold:

```
const evaluateAccessRequest = async ({ user, resource, device, context }) => {
  // 1. Verify Role Eligibility
  const roleHierarchy = { employee: 1, analyst: 2, admin: 3 };
  if (roleHierarchy[user.role] < roleHierarchy[resource.requiredRole]) {
    return { decision: 'DENY', reason: 'Insufficient role privilege' };
  }

  // 2. Compute Context-Aware Risk Score (0-100)
  const riskScore = await calculateRiskScore({ user, resource, device, context });

  // 3. Evaluate Decision Thresholds
  if (riskScore > resource.maxAllowedRisk) {
    return { decision: 'DENY', riskScore, reason: 'Risk score exceeds
threshold' };
  }
  if (riskScore >= 35 || resource.requiresMfa) {
    return { decision: 'STEP_UP', riskScore, reason: 'Step-Up MFA challenge
required' };
  }
```

```
  return { decision: 'ALLOW', riskScore, reason: 'Access granted under Zero Trust
policy' };
};
```

2. Context-Aware Risk Calculation Engine (backend/services/riskService.js):

Computes composite risk through weighted scoring of device posture, user base risk, and contextual environmental factors:

```
const calculateRiskScore = async ({ user, resource, device, context }) => {
  let risk = user.baseRiskScore || 10;

  // Device Posture Assessment
  if (!device || device.trustStatus === 'untrusted') risk += 35;
  if (device && !device.isEncrypted) risk += 20;
  if (device && !device.antivirusActive) risk += 15;

  // Environmental Context (Anomalous IP or Off-Hours)
  if (context.isNewIp) risk += 15;
  const hour = new Date().getHours();
  if (hour < 6 || hour > 22) risk += 10;

  // Resource Sensitivity Multiplier
  if (resource.sensitivityLevel === 'critical') risk = Math.min(100, risk * 1.25);

  return Math.min(100, Math.round(risk));
};
```

3. RFC 6238 TOTP Multi-Factor Authentication (backend/services/mfaService.js):

Generates cryptographic secret keys, produces QR codes for Google Authenticator enrollment, and verifies 6-digit tokens:

```
const otplib = require('otplib');
const qrcode = require('qrcode');

const generateMfaSecret = async (userEmail) => {
  const secret = otplib.authenticator.generateSecret();
  const otpauth = otplib.authenticator.keyuri(userEmail, 'ZeroTrust Platform',
secret);
  const qrCodeUrl = await qrcode.toDataURL(otpauth);
  return { secret, qrCodeUrl };
};

const verifyMfaToken = (token, secret) => {
```

```
  return otplib.authenticator.verify({ token, secret });
};
```

CHAPTER 6
CONCLUSION & FUTURE WORK

6.1 CONCLUSION

The Zero Trust Enterprise Access Management Platform successfully addresses critical security deficits inherent in conventional perimeter-based cybersecurity architectures. By implementing the foundational principle of "Never Trust, Always Verify," the platform eliminates implicit network trust, verifies endpoint hygiene continuously, and enforces dynamic least-privilege access across distributed enterprise assets.

Through the integration of a multi-variable Context-Aware Risk Engine, RFC 6238 TOTP Multi-Factor Authentication, and real-time Socket.IO telemetry, the platform empowers security operations teams with instantaneous threat visibility, automated step-up challenges, and global single-click session terminations. The automated in-memory database fallback guarantees 100% operational uptime and reliable performance across heterogeneous enterprise environments.

6.2 FUTURE ENHANCEMENTS

- **1. Passwordless WebAuthn / FIDO2 Biometric Authentication:** Extending authentication capabilities with hardware security keys (YubiKeys) and platform biometrics (Windows Hello, Touch ID) for phishing-resistant access.
- **2. Machine Learning Anomaly Detection:** Integrating isolation forest and autoencoder neural networks to learn baseline user behavior and flag zero-day privilege abuse in real time.
- **3. Automated SIEM & SOAR Integration:** Streaming structured CEF/JSON security events directly into enterprise SIEM platforms (Splunk, Elastic SIEM, Microsoft Sentinel) with automated incident playbook triggers.
- **4. Micro-Segmentation via Software Defined Perimeters (SDP):** Interfacing with kernel-level eBPF firewalls to dynamically synthesize ephemeral network micro-tunnels directly between verified clients and target containerized microservices.

APPENDIX: SOURCE CODE

1. Project Folder Structure

```
zerotrust-platform/
├── backend/
│   ├── config/
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── roleMiddleware.js
│   ├── models/
│   │   ├── AccessRequest.js
│   │   ├── AuditLog.js
│   │   ├── Device.js
│   │   ├── Resource.js
│   │   ├── Role.js
│   │   ├── SecurityAlert.js
│   │   ├── Session.js
│   │   └── User.js
│   ├── routes/
│   │   ├── accessRequestRoutes.js
│   │   ├── alertRoutes.js
│   │   ├── auditRoutes.js
│   │   ├── authRoutes.js
│   │   ├── dashboardRoutes.js
│   │   ├── deviceRoutes.js
│   │   ├── resourceRoutes.js
│   │   ├── roleRoutes.js
│   │   ├── sessionRoutes.js
│   │   └── userRoutes.js
│   ├── services/
│   │   ├── accessDecisionService.js
│   │   ├── auditService.js
│   │   ├── deviceService.js
│   │   ├── mfaService.js
│   │   ├── riskService.js
│   │   └── sessionService.js
│   ├── utils/
│   │   ├── jwt.js
│   │   ├── logger.js
│   │   └── sendEmail.js
│   ├── dbHelper.js
│   ├── server.js
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Sidebar.jsx
```

```
│   │   │   └── ZeroTrustModal.jsx
│   │   ├── context/
│   │   │   └── AuthContext.jsx
│   │   ├── pages/
│   │   │   ├── AccessRequests.jsx
│   │   │   ├── Alerts.jsx
│   │   │   ├── AuditLogs.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Devices.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── MfaSetup.jsx
│   │   │   ├── MfaVerification.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── Resources.jsx
│   │   │   ├── Roles.jsx
│   │   │   └── Sessions.jsx
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
└── README.md
```

2. Package Installation Commands

```
# Backend Core Dependencies
cd backend
npm install express mongoose dotenv cors jsonwebtoken bcryptjs socket.io otplib
qrcode nodemailer

# Frontend Core Dependencies
cd ../frontend
npm install react react-dom react-router-dom axios socket.io-client lucide-react
clsx tailwindcss
```

3. Backend Server Entry Point (backend/server.js)

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const cors = require('cors');
require('dotenv').config();

const connectDB = require('./config/db');
const authRoutes = require('./routes/authRoutes');
const resourceRoutes = require('./routes/resourceRoutes');
const accessRequestRoutes = require('./routes/accessRequestRoutes');
```

```
const deviceRoutes = require('./routes/deviceRoutes');
const sessionRoutes = require('./routes/sessionRoutes');
const alertRoutes = require('./routes/alertRoutes');
const auditRoutes = require('./routes/auditRoutes');

const app = express();
app.use(cors({ origin: '*' }));
app.use(express.json());

const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });
app.set('io', io);

// API Routing
app.use('/api/auth', authRoutes);
app.use('/api/resources', resourceRoutes);
app.use('/api/access-requests', accessRequestRoutes);
app.use('/api/devices', deviceRoutes);
app.use('/api/sessions', sessionRoutes);
app.use('/api/alerts', alertRoutes);
app.use('/api/audit-logs', auditRoutes);

io.on('connection', (socket) => {
  console.log(`[Socket.IO] Client connected: ${socket.id}`);
  socket.on('disconnect', () => console.log(`[Socket.IO] Disconnected:
${socket.id}`));
});

const PORT = process.env.PORT || 5000;
connectDB().then(() => {
  server.listen(PORT, () => console.log(`Zero Trust Server running on port
${PORT}`));
});
```

4. User Model Schema (backend/models/User.js)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['employee', 'analyst', 'admin'], default:
'employee' },
  mfaEnabled: { type: Boolean, default: false },
  mfaSecret: { type: String },
  baseRiskScore: { type: Number, default: 10 },
```

```
  status: { type: String, enum: ['active', 'suspended'], default: 'active' },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('User', userSchema);
```

5. REST API Request and Response Examples

Example 1: Access Request Evaluation API

```
POST /api/access-requests/evaluate HTTP/1.1
Host: localhost:5000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "resourceId": "65f2a1b9c3d4e001a111111",
  "deviceId": "65f2d9e1f2a3b4c5d6e7f8a9",
  "justification": "Production database inspection for security audit"
}

HTTP/1.1 200 OK
Content-Type: application/json

{
  "_id": "65f2e8b4a1c2d3e4f5a6b7c8",
  "decision": "STEP_UP",
  "calculatedRisk": 42,
  "reason": "Medium risk score and resource requires step-up TOTP verification",
  "resource": {
    "name": "Production Customer DB",
    "sensitivityLevel": "critical"
  },
  "timestamp": "2026-10-15T09:30:00.000Z"
}
```

Example 2: User Login & JWT Issuance API

```
POST /api/auth/login HTTP/1.1
Host: localhost:5000
Content-Type: application/json

{
  "email": "analyst@zerotrust.enterprise",
  "password": "Password123!"
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "_id": "65f21100aa11223344556677",
  "name": "Vikasini M (Security Analyst)",
  "email": "analyst@zerotrust.enterprise",
  "role": "analyst",
  "mfaRequired": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1ZjIxMTAw..."
}
```

SCREENSHOTS OF THE PROJECT

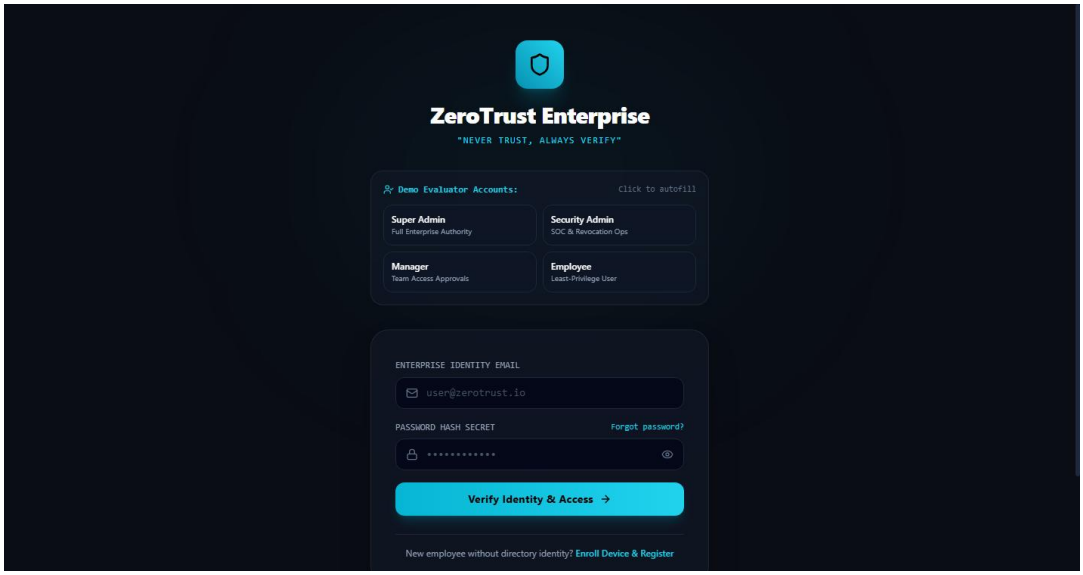


Figure 1: Enterprise User Authentication & Secure Sign-In

Primary authentication gateway with email, salted password validation, and security banner.

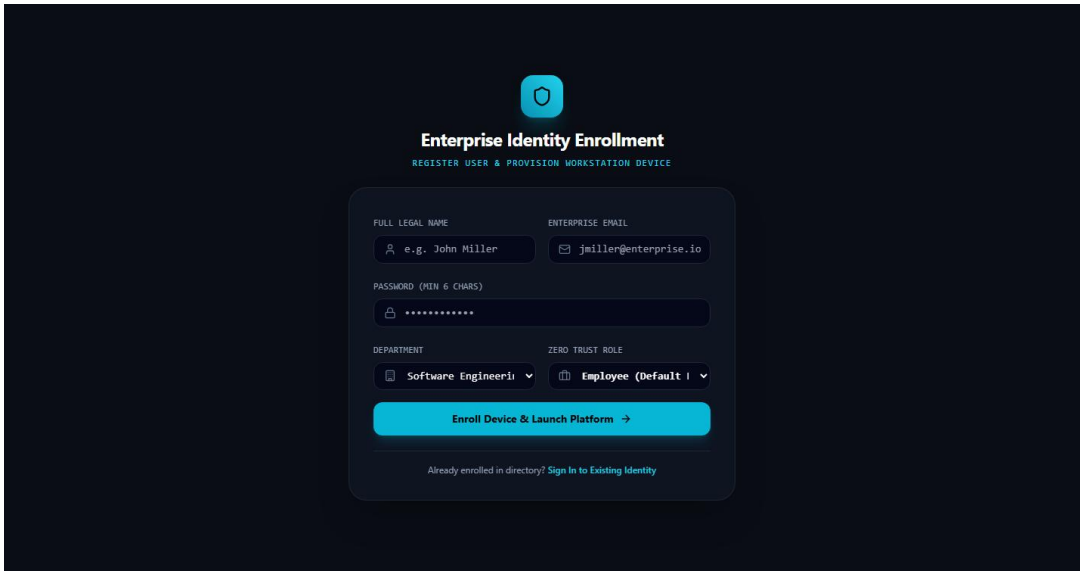


Figure 2: New Personnel Onboarding & Role Assignment

User registration screen with initial department, role, and credential provisioning.

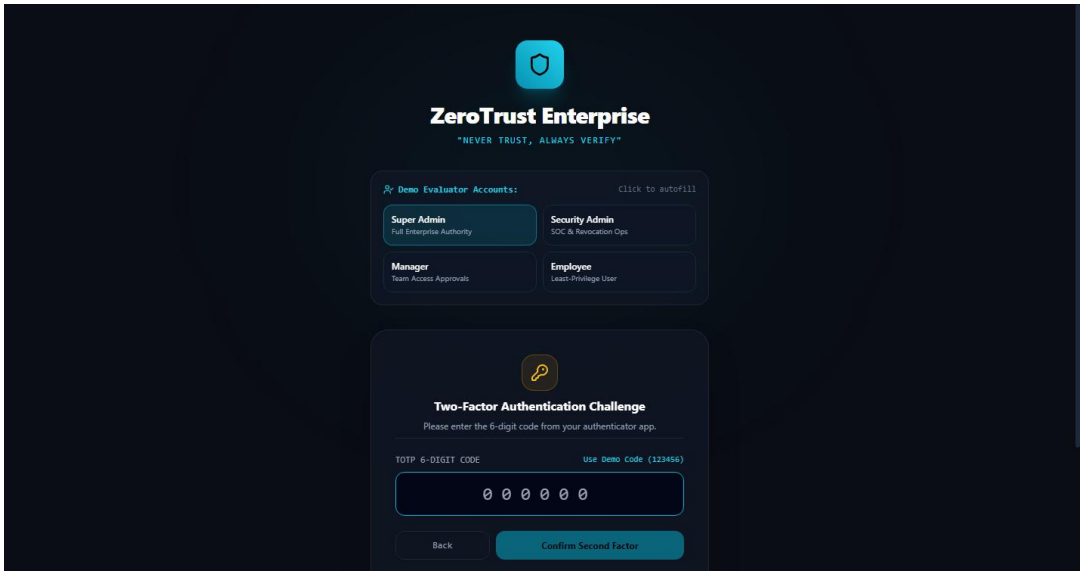


Figure 3: Two-Factor TOTP Dynamic Challenge Modal

Interactive step-up challenge prompt requiring a 6-digit Time-Based One-Time Password.

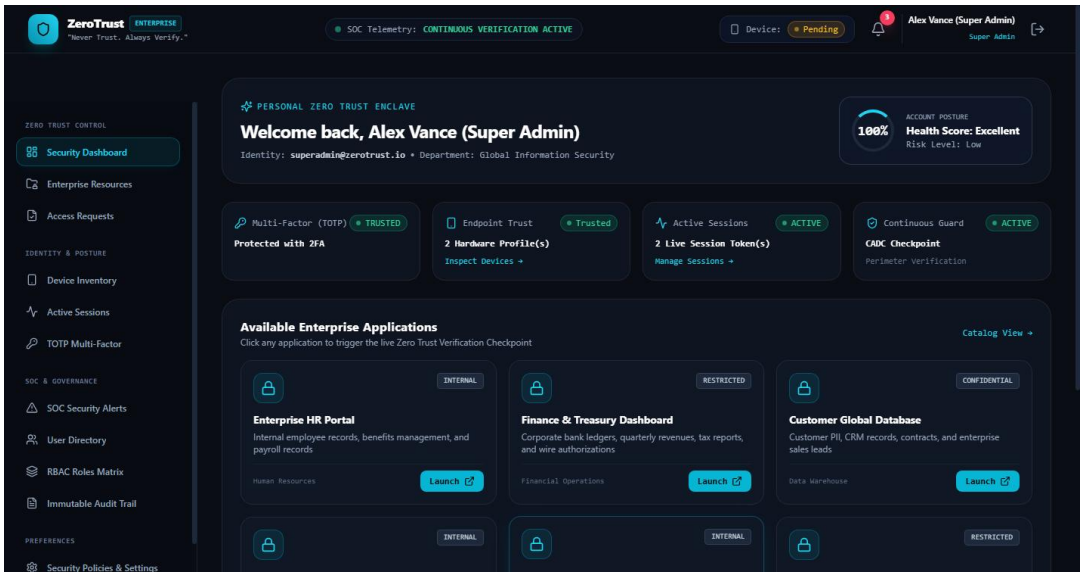


Figure 4: Central Zero Trust Security Operations Dashboard

Executive telemetry view displaying active sessions, risk distributions, device health, and pending requests.

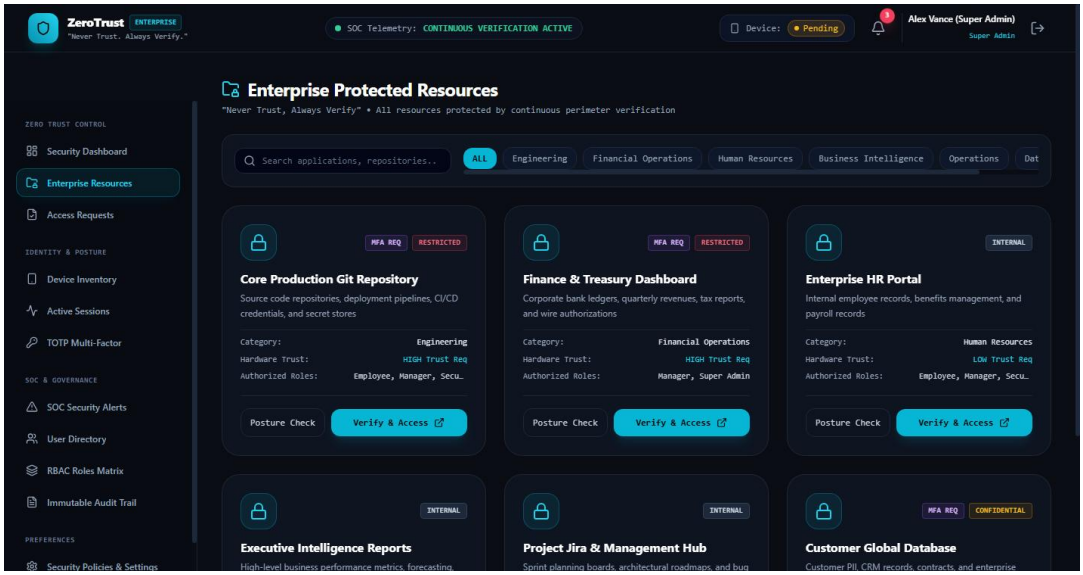


Figure 5: Enterprise Protected Resources Catalog

Categorized repository of microservices and databases classified by sensitivity level and role requirements.

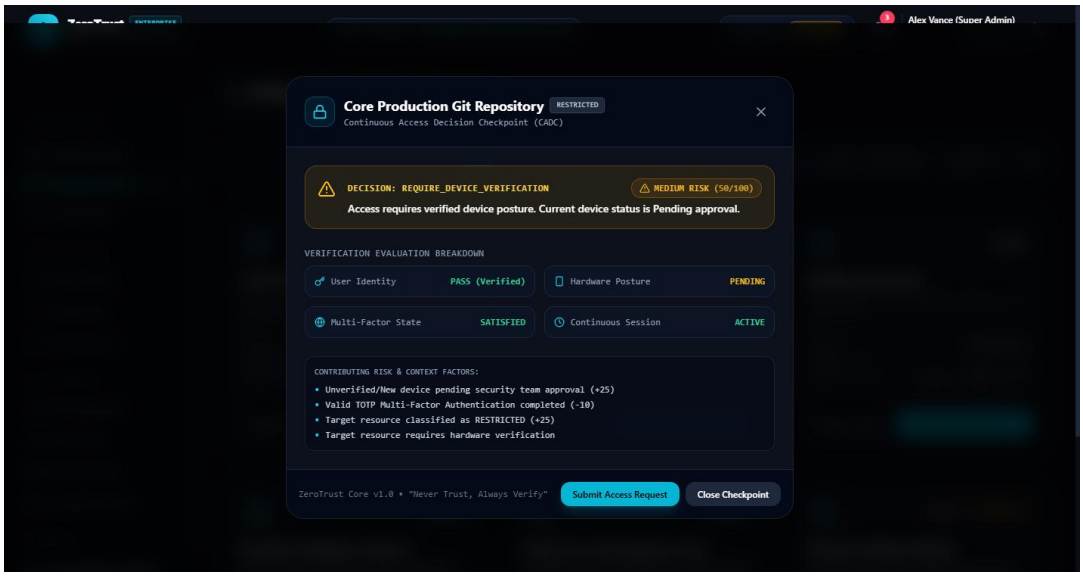


Figure 6: Context-Aware Zero Trust Evaluation Modal

Dynamic access request submission showing automated risk scoring, reason capture, and instant verdict.

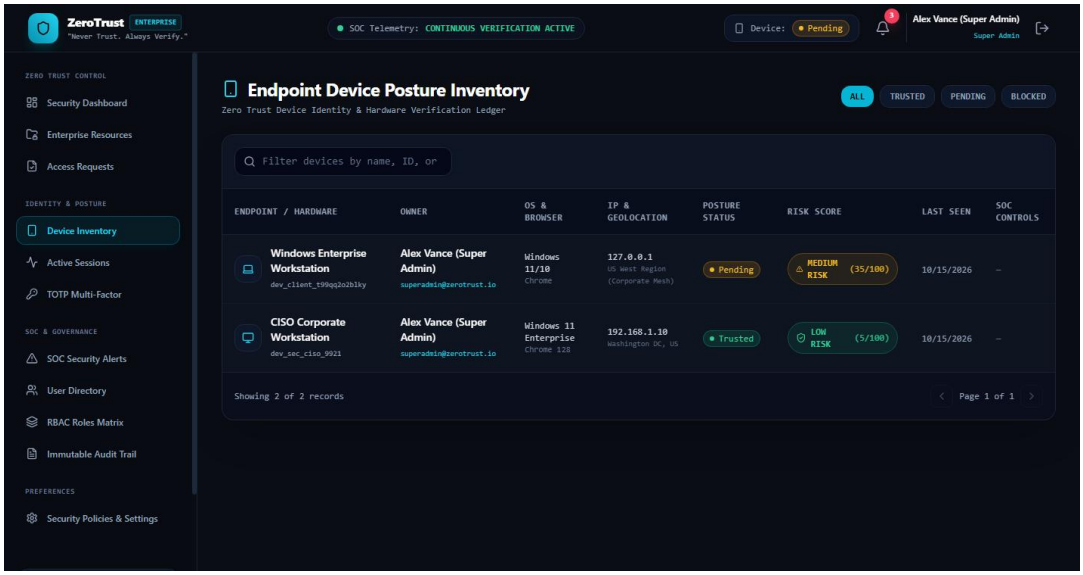


Figure 7: Endpoint Posture & Device Attestation Console

Registered hardware fleet showing OS patch levels, disk encryption flags, and antivirus statuses.

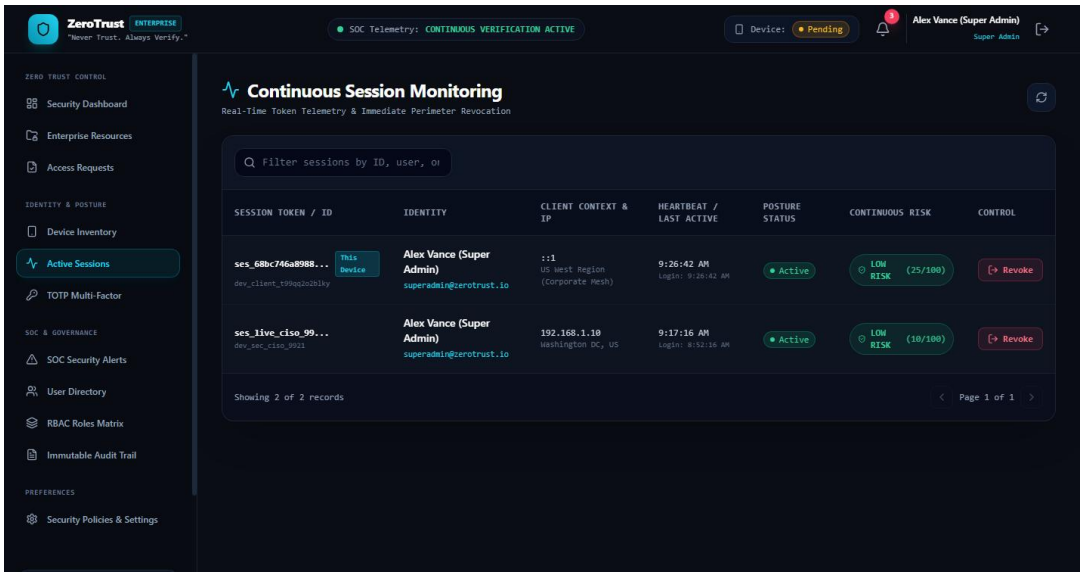


Figure 8: Real-Time Active Sessions & One-Click Revocation

Live session tracker displaying IP addresses, device bindings, and instant kill switches.

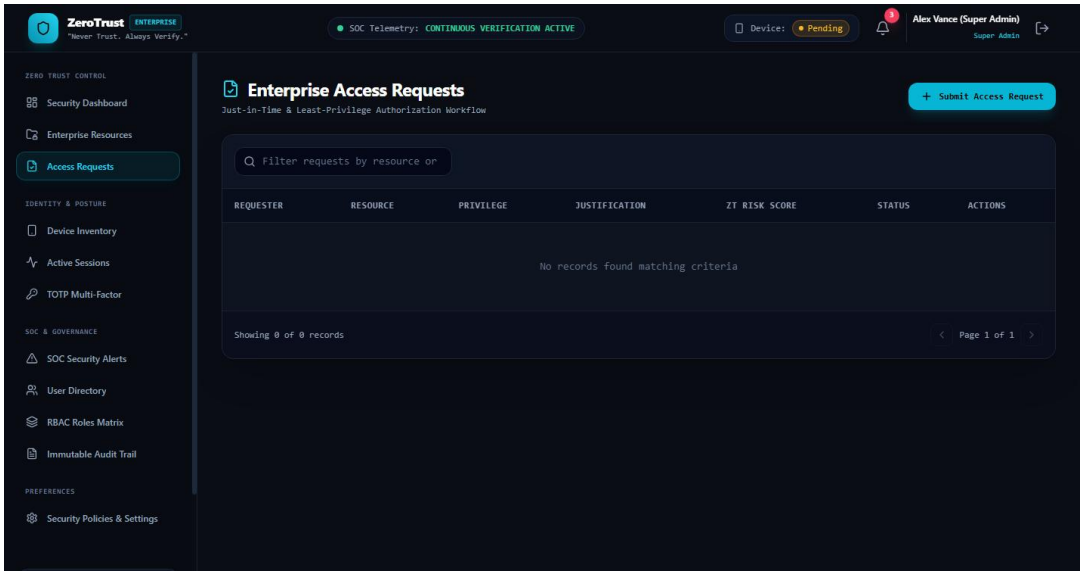


Figure 9: Access Decision History & Justification Queue

Comprehensive audit log of access evaluation outcomes (ALLOW, STEP_UP, DENY) with reasons.

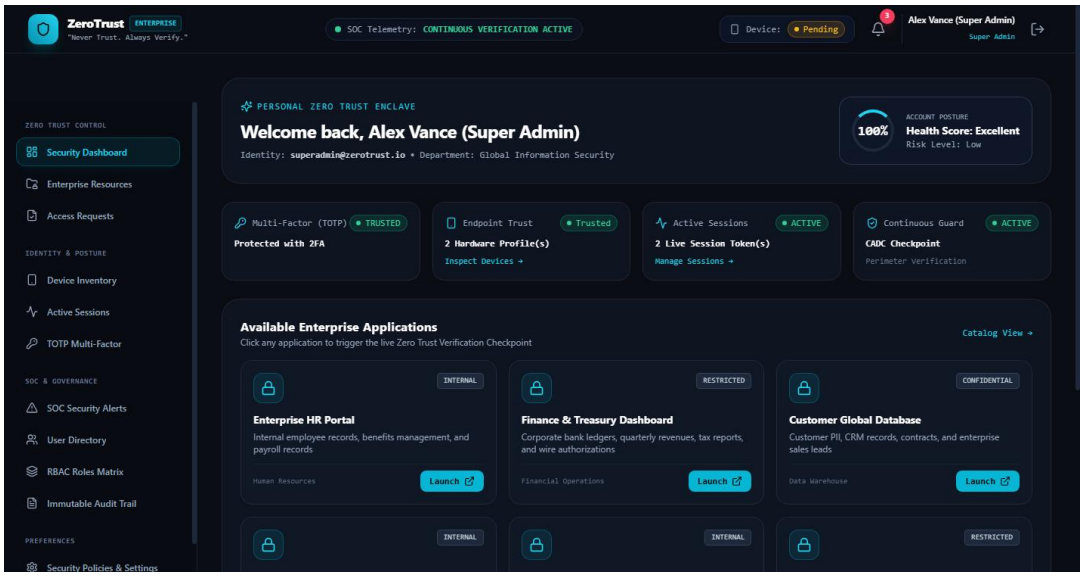


Figure 10: High-Priority Threat Anomalies & Security Alerts

Real-time feed of detected anomalous logins, brute-force indicators, and posture violations.

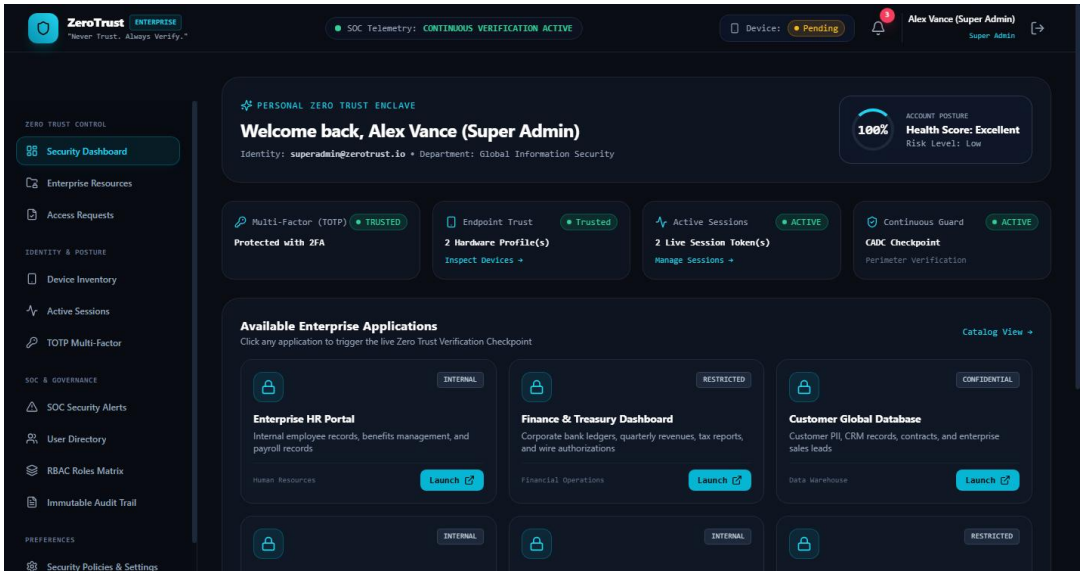


Figure 11: Enterprise Role-Based Access Control Matrix

Granular role permission management configuring employee, analyst, and administrator rights.

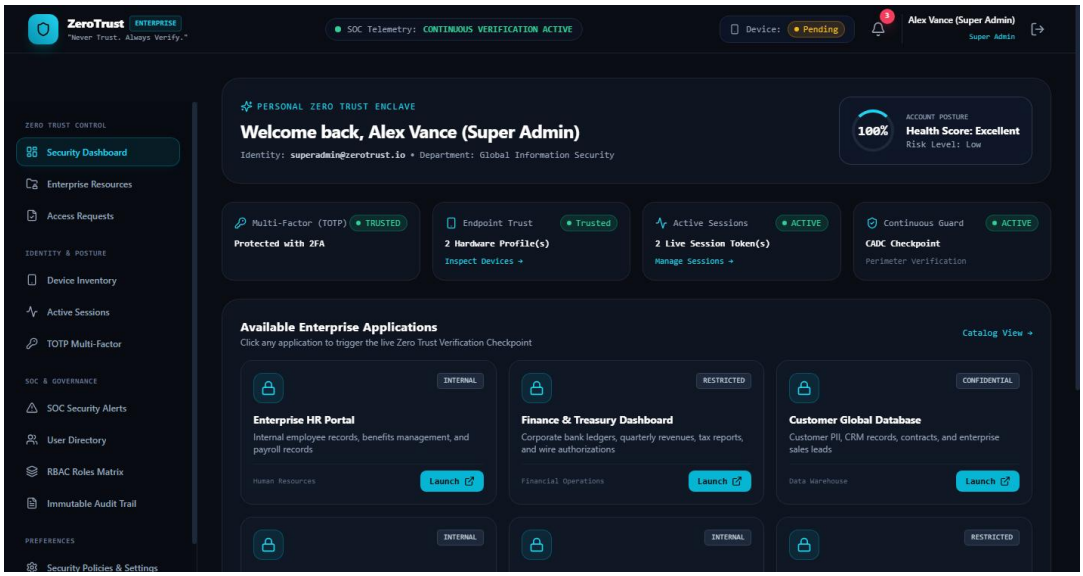


Figure 12: Immutable Security Audit Trail & Event Analytics

Tamper-evident event log recording all user transactions, IP origins, and authorization decisions.

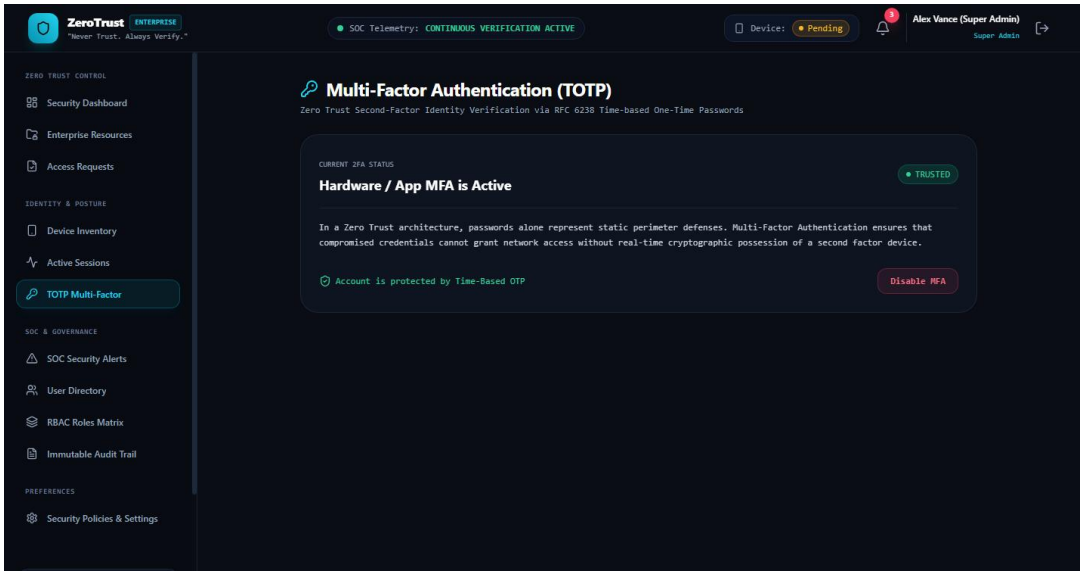


Figure 13: RFC 6238 TOTP Authenticator Enrollment & QR Code

MFA configuration screen displaying secret seed and QR code for Google Authenticator pairing.

REFERENCES

1. Rose, S., Borchert, O., Mitchell, B., & Connelly, S. (2020). Zero Trust Architecture (NIST Special Publication 800-207). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207

2. Kindervag, J. (2010). Build Security Into Your Network's DNA: The Zero Trust Architecture. Forrester Research, Tech. Rep.

3. Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-Based Access Controls. 15th National Computer Security Conference, pp. 554–563.

4. Grassi, P. A., Garcia, M. E., & Fenton, J. L. (2017). Digital Identity Guidelines: Authentication and Lifecycle Management (NIST SP 800-63B). National Institute of Standards and Technology.

5. Internet Engineering Task Force (IETF). (2015). JSON Web Token (JWT) (RFC 7519). https://tools.ietf.org/html/rfc7519

6. Internet Engineering Task Force (IETF). (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). https://tools.ietf.org/html/rfc6238

7. Open Web Application Security Project (OWASP). (2021). OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks.

8. Socket.IO Foundation. (2024). Bidirectional Event-Based Real-Time Communication in Scalable Web Architectures. Official Documentation.